ROS2 AMR 바닥부터 만들기 — 학습자 실습 가이드

> 본 문서는 학습자가 **ROS 2** 환경에서 휠 2개 굴림식 **AMR(Autonomous Mobile Robot)** 을 부품 단위부터 한 단계씩 직접 조립·구동하고, 마지막으로 SLAM 까지 경험하도록 구성한 실습 교재입니다.
> 모든 코드는 **from-scratch(직접 작성)** 를 원칙으로 하되, 표준 ROS 2 패키지는 빌드해서 활용합니다.

목차

- 0. 시작하기 전에
- 부품표 (BOM)
- 좌표계 / TF 트리
- 단계별 실습
  1. ESP32 ↔ PC 시리얼 통신 노드 (PC 디버깅 단계)
  2. Raspberry Pi에서 ESP32 시리얼 통신 + 디버그용 시리얼2
  3. LD14 라이다 — PC에서 RViz로 먼저, Raspberry Pi로 이관
  4. STS3215 서보 PC 제어 노드
  5. PC 통합 테스트 (no-Raspberry Pi)
  6. 전체 조립 + ROS teleop
  7. SLAM
  8. 18650 UPS HAT 전원 시스템
- 부록 A. Pin Map 요약
- 부록 B. 토픽/서비스 정리
- 부록 C. 미검증 항목 / 추가 확인 필요
- 참조 자료

---

0. 시작하기 전에

0.1 안전 주의 (반드시 숙지)

위험	상황	대응
18650 리튬 셀	손상/쇼트 시 발화/폭발	보호회로(PCM/BMS) 내장 셀 사용. 점퍼로 셀 직접 연결 시 퓨즈·반도체 차단 회로 추가. 손상 셀은 즉시 폐기.
STS3215 (및 호환 ST3215) 서보	회전 중 끼임은 고열·소손	첫 동작은**무부하**로. 별도 비상정지 버튼/소프트웨어 stop 명령 마련.
LD14 DToF 라이다	직접 시각 정렬은 적외선	빔은 Class 1 수준이지만 렌즈 표면을 만지지 말 것.
LiPo/UPS HAT 출력	5V/5A	단락 시 누설·과열. 시상 시에는 회로 보호가 있는 USB 전원만 사용.

> 표기사항 출처: BlueRobotics battery info, Waveshare ST3215 wiki

0.2 ROS 2 배포판 결정

본 가이드는 두 LTS(Jazzy/Humble) 모두 동작하게 작성했으나, 예시 명령은 **Jazzy** 를 기준으로 합니다.

ROS 2 Distro	출시	Ubuntu	EOS (서포트 종료)	비고
Humble Hawksbill	2022-05	22.04 LTS	2027-05	본 가이드에서 사용 가능
Jazzy Jalisco	2024-05	24.04 LTS	2029-05	기본 예시
Kilted Kaiju	2025-05	24.04	2026-12	짧은 수명 LTS 아님

출처: ROS 2 Jazzy Releases, ROS 2 EOL tracker(eosl.date)

```
# 분포 확인
echo $ROS_DISTRO
```

0.3 워크스페이스 만들기

```
mkdir -p ~/ros2_amr_ws/src
cd ~/ros2_amr_ws
colcon build --symlink-install
source install/setup.bash
```

> `~/.bashrc` 끝에 `source ~/ros2_amr_ws/install/setup.bash` 를 한 줄 추가하면 새 셸마다 자동으로 source 됩니다.

0.4 필수 apt 패키지

`distro=jazzy` 또는 `humble` 로 변경해 동일하게 실행:

```
sudo apt update && sudo apt install -y \
  python3-pip python3-rosdep python3-colcon-common-extensions \
  ros-$ROS_DISTRO-rviz2 ros-$ROS_DISTRO-teleop-twist-joy \
  ros-$ROS_DISTRO-teleop-twist-keyboard ros-$ROS_DISTRO-joy \
  ros-$ROS_DISTRO-robot-state-publisher ros-$ROS_DISTRO-joint-state-publisher \
  ros-$ROS_DISTRO-xacro ros-$ROS_DISTRO-ros2-control \
  ros-$ROS_DISTRO-ros2-controllers ros-$ROS_DISTRO-slam-toolbox \
  ros-$ROS_DISTRO-nav2-map-server \
  ros-$ROS_DISTRO-ldlidar-sl-ros2 \
  python3-serial
```

0.5 사용자 그룹과 udev 규칙

라이다/서보 IF 보드를 일반 사용자로 다루려면 `dialout` 그룹에 자기 자신을 넣고, **고정 시리얼 포트 이름** udev 룰을 작성합니다.

```
sudo usermod -aG dialout $USER   # 재로그인 필요
```

`/etc/udev/rules.d/99-amr.rules` (예시 — 공급업체 idVendor/idProduct 는 실제 장치에 맞춰 변경):

```
# LD14 LiDAR (CP2102 가정)
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", SYMLINK+="lidar_port"
# Waveshare Serial Bus Servo Driver Board (CH343 예시 — 확인 필요)
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="55d3", SYMLINK+="servo_port"
# ESP32 USB (디버깅용)
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", SYMLINK+="esp32_debug"
```

```
sudo udevadm control --reload-rules && sudo udevadm trigger
ls -l /dev/lidar_port /dev/servo_port /dev/esp32_debug
```

> `idVendor/idProduct` 는 `lsusb` 로 확인 후 **반드시 본인 환경의 값으로 교체**하세요.

부품표 (BOM)

> 모든 행의 [✓] 항목은 본 가이드 작성 시점에 독립 검증을 한 부분입니다.
> 검증되지 않은 항목은 **[확인 필요]** 표시를 했습니다.

#	부품	수량	역할	전원 / 전압 비고	검증 / 출처
1	Raspberry Pi 5 (또는 4B)	1	메인 ROS2 호스트	UPS HAT 통한 5V(=5A max)	Waveshare UPS HAT
2	ESP32 DevKit V1 (WROOM-32)	1	서보/Motor low-level 컨트롤러	5V USB 또는 3.3V 라인	UART2 = GPIO16/17 (ESP32 pinout by RandomNerd)
3	STS3215 (Feetech, ST3215 호환)	2	좌/우 휠 직접 구동 (continous-rotation wheel mode)	6–12.6V (12V 권장)	DFRobot ST3215, Waveshare ST3215 wiki
4	**STS IF 보드** (Waveshare Serial Bus Servo Driver Board)	1	half-duplex 단선 ↔ USB 가상 COM 변환, 12V 입력 DC 잭	12V(서보 전원), 5V(USB)	Waveshare Bus Servo Driver Board wiki **[확인 필요: 제품 SKU는 구매처 라벨로 재확인]**
5	MPU6050 (GY-521 등 브레이크아웃)	1	IMU(자세 추정)	3.3V	I²C addr 0x68(GND), 0x69(AD0=HIGH) (ControllersTech MPU6050)
6	LD14 DToF LiDAR (LDROBOT 360°)	1	2D laser scan	5V(USB-to-serial 모듈 경유)	ldrobotSensorTeam/ldlidar_sl_ros2
7	18650 배터리 팩 (3.7V 공칭, 보호회로 있는 셀)	2	UPS HAT에 장착	4.2V 만충 / 3.0V 차단	Large-battery 18650 guide
8	USB-to-Serial (CP2102 / CH340)	1	라이다/디버깅용	–	ldlidar_sl_ros2 README - USB to serial
9	0.96" OLED (SSD1306, I²C)	1	상태 표시	3.3V 또는 5V	addr 0x3C 가**기본값** (확인 필요: 모듈 일부 0x3D)
10	18650 UPS HAT (Raspberry Pi 4B/3B+/5)	1	깔끔한 모바일 5V 전력 + UPS	5V/최대 5A 출력	Waveshare UPS HAT
11	점퍼 와이어/브레드보드/하우징(chassis)	-	배선/모음	–	–

0.x IPU / "IPU"에 관한 메타 메모 (검증 필요)

원 작성에서 언급된 **"IPU"** 가 정확히 어떤 약어/노드인지 본 가이드 단계에서는 확정할 수 없습니다.
따라서 본 문서에서는 `cmd_vel` (geometry_msgs/msg/Twist) 와 같은 **표준 제어 흐름** 으로 읽고 학습하게 코드를 작성합니다. 상호작용 모듈의 입출력 정의는 클래스 단위로 배우기 쉽도록 분리하고, 멘토가 명확한 정의(예: `IPU = Integrated Power Unit`, 또는 사용자 정의 모듈)를 알려주는 경우에만 교체하도록 했습니다.

좌표계 / TF 트리

드디어 차량 좌표계를 정의합니다.

```
map
 └── odom
      └── base_link
           ├── wheel_left_link   (STS3215 #1)
           ├── wheel_right_link  (STS3215 #2)
           ├── laser_frame       (LD14, base_link 위로 z=0.20m)
           └── imu_link          (MPU6050, base_link 중심)
```

URDF (Xacro) 발췌

`src/amr_description/urdf/amr.xacro`:

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="amr">
  <xacro:property name="wheel_radius" value="0.065"/>
  <xacro:property name="wheel_sep"    value="0.30"/>

  <!-- base_link -->
  <link name="base_link"/>
```

```
  <joint name="base_to_imu" type="fixed">
    <parent link="base_link"/><child link="imu_link"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
  </joint>
  <link name="imu_link"/>

  <!-- wheels (continuous joints for diff-drive) -->
  <joint name="base_to_wheel_left" type="continuous">
    <parent link="base_link"/><child link="wheel_left_link"/>
    <axis xyz="0 0 1"/>
    <origin xyz="0  ${wheel_sep/2}  0" rpy="0 0 0"/>
  </joint>
  <link name="wheel_left_link"/>

  <joint name="base_to_wheel_right" type="continuous">
    <parent link="base_link"/><child link="wheel_right_link"/>
    <axis xyz="0 0 1"/>
    <origin xyz="0 -${wheel_sep/2}  0" rpy="0 0 0"/>
  </joint>
  <link name="wheel_right_link"/>

  <!-- laser -->
  <joint name="base_to_laser" type="fixed">
    <parent link="base_link"/><child link="laser_frame"/>
    <origin xyz="0.10 0 0.20" rpy="0 0 0"/>
  </joint>
  <link name="laser_frame"/>
</robot>

ros2 run xacro xacro src/amr_description/urdf/amr.xacro > /tmp/amr.urdf
ros2 run robot_state_publisher robot_state_publisher --ros-args -p robot_description:="$(cat /tmp/amr.urdf)"
```

단계별 실습

1단계: ESP32 ↔ PC 시리얼 통신 노드 (PC 디버깅 단계)

이 단계는 **PC ↔ ESP32 단기 loop** 단독 테스트입니다. 라즈베리파이는 사용하지 않고, 시리얼 디버거를 PC에서 직접 열어 메시지를 눈으로 확인합니다.

1.1 준비물

- ESP32 DevKit V1 (USB-C/Micro-B)
- USB-to-Serial 모듈(CP2102) 또는 ESP32 자체 USB-UART
- 점퍼 와이어 3개 (GND, TX, RX) — ESP32 ↔ CP2102

1.2 배선

> ESP32는 USB 라인(UART0, GPIO1 ↔ GPIO3)을 **점유** 하고 있으므로 디버깅에는 USB 단자를 그대로 쓰는 게 가장 쉽습니다.

ESP32 쪽	PC 쪽 (CP2102 사용 시)
GND	GND
GPIO1 (TX0) ← USB-UART BRIDGE 출력 ← **RX/DTR 불필요**	ESP32 USB-C 라인 사용 시에는 내부적으로 자동 연결
GPIO3 (RX0)	동일

기본 UART 핀: `UART0 TX=GPIO1 RX=GPIO3` (PCBSync ESP32 pinout ref).

1.3 프로토콜 정의

`Serial.begin(115200, SERIAL_8N1)` — 디버그 단계에서는 **115200** 권장 (서보용 1 Mbps 와 분리). 한 줄 단위 ASCII:

```
PC → ESP32:  $cmd,linear_x,angular_z\n
ESP32 → PC:  $status,voltage_mV,pose_ok\n
```

1.4 ESP32 Arduino 스케치 (`esp32_bridge.ino`) — 의도와 구조

이 스케치는 **PC/라즈베리파이에서 들어오는 제어 명령을 두 글로벌 변수 g_v, g_w(각각 m/s, rad/s)로 저장하고, 0.5 s 마다 진단 라인을 한 줄 echo** 하는 최소 루프입니다. 1단계의 목표는 단일 라운드트립(PC→ESP32→PC)을 검증하는 것이므로 모터 구동 로직은 일절 포함하지 않습니다. 학생은 이 단계에서 "시리얼 라인 프로토콜" 이 무엇인지 몸으로 익히게 됩니다.

무엇을 왜 만드는가 — 핵심 설계 의도

- **부트로더 겸용 핀(UART0 = GPIO1/GPIO3)을 USB-UART 출력으로 그대로 사용**합니다. 디버그 단계의 모든 메시지가 USB-Serial 모니터에 단일 채널로 보이도록 단순화합니다 — 학생이 `minicom/screen` 으로 즉시 확인할 수 있게 하려는 의도입니다.
- **ASCII 라인 프로토콜** (`$cmd,linear_x,angular_z\n` ↔ `$status,...\n`) 을 선택했습니다. 이진 패킷보다 손으로 디버깅하기 쉬워서 라이다·IMU 가 추가되는 단계 3·5 에서도 같은 형태로 확장되기 때문입니다.
- **입력 파싱(`parseLine`)과 액추에이션(`loop`)을 분리**합니다. 명령은 들어오든 안 들어오든 500 ms 주기로 `$status,uptime_ms=…` 진단 신호를 일정하게 흘려보내고, 명령이 폭주해도 출력 폭주가 일어나지 않게 만듭니다.
- **Soft-estop** : `$cmd,0,0` 명령이 들어오면 자연스럽게 0 속도로 안정화되도록 의도합니다. 비상정지를 별도 핀/토픽으로 두지 않고 같은 라인으로 흡수하는 것이 학습 부담을 낮춥니다.

구현 시 반드시 챙길 디테일

- `Serial.begin(115200, SERIAL_8N1)` 로 baud·데이터비트·패리티를 통일합니다. 한 곳만 어긋나도 첫 문자부터 깨집니다.
- `setup()` 끝에 `# esp32_bridge ready @115200` 배너를 한 줄 출력해, 플래시·USB 연결이 정상인지를 즉시 확인하게 합니다.
- `loop()` 안의 `static String buf` 는 누적 버퍼로 둡니다 — `delay()` 로 한 글자씩 묶지 않기 위함입니다. 같은 패턴이 추후 단계의 다른 모듈에서도 등장합니다.

- 명령 파싱 직후 `$ack,v=…,w=…` 라인을 즉시 echo 합니다 — PC 측에서 "명령이 들어갔는지" 를 즉시 검증 가능하게 합니다.

직접 구현 체크리스트 (학생용)

- ☐ `Serial.begin()` 은 `setup()` 안에만 둔다.
- ☐ `$acctrect` 의 소수점 정밀도는 `print(value, 3)` 으로 3자리.
- ☐ `millis()` 기반 주기 처리는 `static` 변수로 last-time 기억. `delay()` 사용 금지.
- ☐ 모든 송신 라인은 `\n` 으로 끝맺는다.
- ☐ 어떤 명령에도 `$ack,…` reply 가 반드시 가도록 보장한다.

1.5 ROS 2 Python 노드 (`amr_serial/amr_serial/esp32_bridge.py`) — 의도와 구조

이 노드는 **ROS 2 의 표준 메시지(`/cmd_vel`) 를 ESP32 의 ASCII 라인 프로토콜로, 그리고 그 반대로 변환하는 "양방향 어댑터"** 입니다. 직접 작성할 때의 학습 목표는 `rclpy` 노드의 표준 구조(파라미터 → Publisher/Subscriber 생성 → 타이머) 를 몸으로 익히는 것이고, 단계 5·6·7 에서 모든 노드를 같은 골격으로 짜게 됩니다.

노드의 I/O 계약

- **입력 토픽**: `/cmd_vel` (`geometry_msgs/Twist`, linear.x = m/s, angular.z = rad/s)
- **출력 토픽**: `/amr/status` (`std_msgs/String`, ESP32 가 보낸 한 줄을 그대로 echo)
- **시리얼 입력/출력**: 동일 115200 baud 의 한 줄 ASCII — ESP32 Arduino 스케치와 정확히 같은 라인을 사용
- **노드 파라미터**: `port`(`/dev/esp32_debug` 또는 RPi 에서는 `/dev/ttyAMA0`), `baud`(115200), `wheel_sep`(m), `wheel_radius`(m)

무엇을 왜 만드는가 — 핵심 설계 의도

- **subscription 콜백에서 "즉시 송신" 하지 않고 target 변수만 갱신**합니다. 콜백에서 직접 시리얼을 쓰면 (a) 메시지 폭주 시 버퍼가 점유되고 (b) 단계 7의 SLAM 이 발행하는 `/cmd_vel` 재출력(주:SLAM 의 회귀 경로) 과 매끄럽게 합쳐지지 않습니다. 두 단계에서 "target_v/target_w 상태변수 → 50 Hz 타이머 → 송신" 패턴이 표준이 됩니다.
- **속도 상한(soft e-stop) 을 subscription 단계에 둡니다.** 조이스틱이 비정상적인 값을 흘려보내도 ESP32 가 받는 값이 항상 [-2 m/s, +5 rad/s] 이내로 잘려 들어가는 1차 방어선입니다. 4단계의 STS3215 컨트롤러가 받는 속도 범위(±32767 step/s)와 같은 규모로 잘라 두면 단계 6·7 의 토픽 매핑이 단순해집니다.
- **수신은 best-effort** — `in_waiting` 으로 남아 있는 바이트가 있을 때만 한 줄을 읽어 `/amr/status` 로 그대로 흘려보냅니다. ROS 토픽으로 다시 송출하는 이유는 dashboard RViz 등 다른 도구에서 "ESP32 가 무엇을 보고 있는가" 를 GUI 로 확인하기 위함입니다.
- **타이머 주기 0.02 s = 50 Hz** : 차륜형 AMR 의 제어 루프치고는 느슨하지만, 시리얼 한 줄 송수신 + ASCII 라인 처리에는 충분히 빠른 주기입니다. 더 높이면 오히려 UART FIFO 가 막힙니다.

흐름도 (요약)

1. `init` 에서 파라미터 4개 선언 → `serial.Serial` 엽니다.
2. `cmd_vel` subscription 콜백은 target 값을 갱신만 합니다.
3. 50 Hz 타이머 콜백에서 매 틱마다 `target_v/target_w` 를 한 줄 ASCII 로 송신하고, 들어와 있는 응답은 ROS 토픽으로 다시 송출합니다.
4. 시리얼 송수신 실패는 warn 로그로 남기고 continue — 노드는 절대 죽지 않게 설계합니다 (PC ↔ RPi 분리 학습에서 흔한 "노드 죽음 → 다시 launch" 를 피하기 위함).

직접 구현 시 꼭 챙길 함정 (이전 결함 메모)

1. **같은 토픽을 publisher 와 subscription 에 동시에 두지 마세요.** "디버그 echo 를 위해 자기가 publish 한 것을 다시 subscribe" 하는 패턴은 무한 루프를 만듭니다. 디버그 echo 토픽명은 분리하세요 (`/amr/status` ↔ `cmd_vel`).
2. **TF 프레임은 한 가지 명칭으로 통일**합니다. 본 가이드에서는 `laser_frame` 으로 통일했으니 URDF 와 LiDAR launch 의 `frame_id=` 인자 모두 `laser_frame` 으로 맞추세요.
3. **`setup.py` 의 `entry_points` 누락은 가장 흔한 실수** — `colcon build` 가 성공해도 `ros2 run amr_serial esp32_bridge` 가 "executable not found" 로 죽습니다. 부록 D 의 골격을 사용하면 안전합니다.
4. **liDAR launch 의 `frame_id=` 를 명시** : `ldlidar_sl_ros2` 의 기본값은 `laser` 이라 URDF 와 다릅니다. launch 인자로 `frame_id:=laser_frame` 을 반드시 넘깁니다.

직접 구현 체크리스트 (학생용)

- ☐ `declare_parameter` 4 개와 `get_parameter(...).value` 캐싱이 동일하게 호출되는가.
- ☐ subscription 콜백에서 target 변수를 갱신만 하고 직접 송신하지 않는다.
- ☐ 50 Hz 타이머는 `create_timer(0.02, …)` 로 만들고 콜백 안에서 예외는 warn 로그만 남긴다.
- ☐ 시리얼 read 시 `try/except` 와 `decode('ascii','ignore')` 로 한 바이트 깨짐에도 죽지 않게 한다.
- ☐ 종료 핸들러에서 `serial.close()` 를 호출해 다음 launch 에서 "port already open" 이 나지 않도록 한다.

`package.xml`:

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>amr_serial</name>
  <version>0.0.1</version>
  <description>ESP32 serial bridge for AMR (cmd_vel ↔ ASCII line protocol)</description>
  <maintainer email="edu@example.com">Edu</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_python</buildtool_depend>
  <depend>rclpy</depend>
  <depend>geometry_msgs</depend>
  <depend>std_msgs</depend>
  <exec_depend>python3-serial</exec_depend>
  <export><build_type>ament_python</build_type></export>
</package>
```

`setup.py` (colcon 빌드 후 `ros2 run amr_serial esp32_bridge` 가 동작하려면 **entry_points** 필수):

```
from setuptools import setup
package_name = 'amr_serial'
setup(name=package_name, version='0.0.1',
      packages=[package_name],
```

```
      data_files=[('share/ament_index/resource_index/packages', ['resource/' + package_name]),
                  ('share/' + package_name, ['package.xml'])],
      install_requires=['setuptools', 'pyserial'],
      zip_safe=True, maintainer='Edu', maintainer_email='edu@example.com',
      description='ESP32 serial bridge for AMR', license='MIT',
      entry_points={
          'console_scripts': [
              'esp32_bridge = amr_serial.esp32_bridge:main',
          ],
      },
)
```

1.6 실행 및 검증

```
# 새 터미널
ros2 run amr_serial esp32_bridge --ros-args -p port:=/dev/ttyUSB1 -p baud:=115200

# 또 다른 터미널
ros2 topic pub --rate 5 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.2}, angular: {z: 0.0}}'
```

검증 체크리스트:

```
ros2 topic list                      # /cmd_vel, /amr/status 표시 여부
ros2 topic info /amr/status          # publishers: 1, subscribers: 0 (정상)
ros2 node info /esp32_bridge         # PID 표시되어야 함
```

ESP32 가 ack 를 찍으면 성공. `ros2 topic echo /amr/status` 로 `$status,...` 가 흘러야 합니다.

1.7 트러블슈팅

증상	원인/대응
Permission denied: '/dev/ttyUSB1'	① 사용자를 `dialout` 에 추가 후 재로그인 ② udev 룰로 본인 uid 가 RW 가능하도록
깨진 문자(????)가 출력됨	baud 불일치: 양쪽 모두 같은 `115200 8N1` 인지 검증
ESP32 가 가끔 재셋됨	USB 케이블 굵기 부족 (특히 5V 전력 라인)
`Port already open`	부트로더 모드 진입 후 점유 —`dmesg` 로 `cp210x converter now attached` 후 재시도

2단계: Raspberry Pi에서 ESP32 시리얼 통신 + 디버그용 시리얼2

여기서는 USB-시리얼 대신 **라즈베리파이 GPIO UART** 와 ESP32 의 **UART2 (GPIO16/17)** 를 직접 연결합니다. 동시에 별도 디버깅용 **시리얼 2번** (=USB-to-Serial 모듈 경유) 을 두어 RPi 만 돌릴 때 PC에서 로그/명령을 같이 볼 수 있도록 합니다.

2.1 준비물

- RPi 40-pin GPIO ↔ ESP32 점퍼 4개 (GND, TX, RX, +5V 옵션)
- CP2102/CH340 USB-to-Serial 모듈 (RPi ↔ ESP32 라인 위에서 디버깅 처리)
- 위 1단계의 코드/Launch 파일 그대로

2.2 배선

RPi (BCM 14/15)	ESP32 (UART2)	CP2102
GND (pin 6)	GND	GND
GPIO14 / TX (pin 8)	GPIO16 / RX2	–
GPIO15 / RX (pin 10)	GPIO17 / TX2	–
–	–	CP2102 RX ← ESP32 GPIO17, TX ← GPIO16 (디버깅 T-탭으로 보기)

ESP32 UART2 기본 매핑은 `TX2=GPIO17 RX2=GPIO16` (RandomNerd Tutorials pinout).

> ESP32 의 **UART1** 은 GPIO9/10 이며 **플래시 점유** 라 사용 비권장.
> 따라서 2차 "디버깅용 시리얼" 은 (a) UART2 의 T-탭 라인에서 살짝 도청하든, (b) 추가로 SoftwareSerial(검증 필요)을 쓰든, (c) ESP32 UART2를 메인으로 두고 별도 GPIO를 "SoftSerial"로 디버깅용 보로 만들어도 됩니다. 본 가이드 **(a) 안** + 안전 GPIO에 메인 UART를 옮기는 절충안을 권장합니다.
> **확인 필요:** ESP32 부트로더가 UART0 를 사용하므로 디버깅용 시리얼은 UART0 가 아니라 **UART2/안전 GPIO SoftSerial** 을 사용해야 합니다.

2.3 RPi UART 활성화

`/boot/firmware/config.txt` (Pi 5) 또는 `/boot/config.txt` (Pi 4) — RPi 5 일 경우 벌써 활성화돼 있을 수 있음:

```
enable_uart=1
dtoverlay=disable-bt                # HCI UART 를 GPIO 라인으로 끌어옴

sudo systemctl disable --now hciuart
sudo reboot
ls -l /dev/ttyAMA0 /dev/ttyS0 2>/dev/null
```

2.4 launch 구성

`src/amr_bringup/launch/rpi_esp_bridge.launch.py`:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='amr_serial',
            executable='esp32_bridge',
            output='screen',
```

```
            parameters=[{
                'port': '/dev/ttyAMA0',
                'baud': 115200,
                'wheel_sep': 0.30,
                'wheel_radius': 0.065,
            }],
            remappings=[('cmd_vel', 'cmd_vel')],
        ),
    ])
```

2.5 디버깅용 시리얼 2(USB-to-Serial) 동시 로그

PC 쪽에서 ESP32 UART2 라인에 T-탭으로 CP2102 연결 (디버깅 전용 listen-only):

```
# 두 시리얼(ESP32 ↔ RPi, PC ↔ ESP32) 동시에 보기
minicom -b 115200 -D /dev/ttyUSB1
# RPi 에서 cmd 보낼 때
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist '{linear:{x:0.1},angular:{z:0.2}}'
```

> **확인 필요:** ESP32 의 RX2 라인이 RPi-공유 라인이라면 같이 TX 함. 본 구성은 단순 도청(공유 TX)형, RPi → ESP32 도 같이 봄.

2.6 트러블슈팅

증상	원인
`/dev/ttyAMA0` 가 안 보임	`enable_uart=1` 미적용 또는 overlay 문제
데이터가 깨짐	공통 GND 누락 / 케이블 길이가 너무 길어 잡음
ESP32 라인이 멈춤	USB 케이블을 뽑았다가 꽂으면 자동 재셋 — 의도치 않게 부트로더 진입 가능 (RST→EN 핀 수동 reset 회로 권장)

3단계: LD14 라이다 — PC에서 RViz로 먼저, Raspberry Pi 로 이관

3.1 준비물

- LD14 LiDAR (LDROBOT DToF 360°) — 보조 USB-to-Serial 모듈 (CP2102) 필요
- 안정적인 5V/USB 전원

3.2 PC 에서 연결

```
# LiDAR PH2.0-4P → CP2102 RX/TX/5V/GND
# CP2102 USB ↔ PC
ls -l /dev/lidar_port
```

3.3 패키지 빌드

```
cd ~/ros2_amr_ws/src
# 1) apt 로 설치 (apt pkg 가 distro에 있는 경우)
sudo apt install -y ros-$ROS_DISTRO-ldlidar-sl-ros2

# 2) 소스에서 직접 빌드 (apt 없을 때에만)
git clone https://github.com/ldrobotSensorTeam/ldlidar_sl_ros2.git -b master
cd ~/ros2_amr_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install --packages-select ldlidar_sl_ros2
source install/setup.bash
```

출처: ldlidar_sl_ros2 README (`ld14.launch.py` 가 115200 baud 기준).

3.4 launch

```
# frame_id 는 xacro 의 laser_frame 과 일치하도록 명시 override (확인 필요: ldlidar launch 인자 명은 distro 에 따라 'frame_id'/'laser_frame_i
ros2 launch ldlidar_sl_ros2 ld14.launch.py \
  port_name:=/dev/lidar_port \
  frame_id:=laser_frame \
  laser_scan_dir:=True

ros2 topic hz /scan
# ≈ 10~12 Hz 이어야 정상
ros2 topic echo /scan --field header.frame_id
# → "laser_frame" 출력 확인
```

3.5 RViz2 에서 시각 확인

```
rviz2
```

- Fixed Frame: `laser_frame`
- Add → `LaserScan` (Topic: `/scan`)
- Size(m): `0.05`, Style: `Points`

3.6 RPi 이관

RPi 에서 같은 패키지 설치/빌드, 위 launch 그대로 실행.

3.7 트러블슈팅

증상	점검
`/scan` 안 잡힘	①`serial_baudrate=115200` 인지, ② RX/TX 교차, ③ 5V 전원 안정
거꾸로 그려짐	`laser_scan_dir:=True/False` 토글 (ldlidar_sl_ros2 README)

증상	점검
`frame_id` 미스매치	`/scan` Header 의 `frame_id=laser_frame` 으로 — xacro의 `laser_frame` 과 일치해야
RViz 가 검은 화면	LaserScan 의 `Size(m)` 너무 작거나 Fixed Frame 오타

4단계: STS3215 서보 PC 제어 노드

4.1 STS3215 기본 사양 (출처: DFRobot product page, Indystry STS3215 review)

- 프로토콜: Feetech STS 시리즈 — **half-duplex UART**, 0..4095 위치 단위
- Baud: **1 Mbps 기본값**, 38400 ~ 1 Mbps 지원
- ID: 1..253
- 응답: 위치/온도/전압/부하

4.2 IF 보드 활용

Waveshare Serial Bus Servo Driver Board (wiki) 이용.
USB-C 연결 → PC에 CDC-ACM 가상 COM 포트로 등장(`/dev/servo_port`).

- 보드의 **DC 5.5×2.1 잭** 에 12V(서보용) 입력
- 보드의 **3-pin 단자** (Signal/V/GND) → STS3215 신호 라인 (단선 half-duplex)
- 보드 자체에 half-duplex 변환 회로가 있어 TX/RX 합선이 자동으로 처리됨 (eevblog Waveshare servo driver thread)

4.3 PyPI 패키지

가장 단순한 옵션은 `st3215` PyPI 패키지 — Feetech 공식 문서/예제에 준거.

```
pip3 install st3215 pyserial
```

4.4 ROS 2 노드 (`amr_servo/amr_servo/sts_controller.py`) — 의도와 구조

이 노드는 **차륜형 AMR 의 좌/우 휠 속도를 ROS 2 메시지(`/cmd_vel` 또는 `/left_wheel/cmd`·`/right_wheel/cmd`) 로 받아서 STS3215 의 wheel-mode 명령으로 변환·송출하는 컨트롤러** 입니다. 1단계의 ESP32 브리지와 마찬가지로 input 갱신 단계와 실제 송신 단계를 분리한 것이 핵심 골격이며, 단계 5·6 에서 모든 휠 기반 노드를 같은 패턴으로 확장합니다.

노드의 I/O 계약

- **입력 토픽 3가지**
  - `/cmd_vel` (`geometry_msgs/Twist`) — 키보드/조이스틱 teleop 출력
  - `/left_wheel/cmd`, `/right_wheel/cmd` (`std_msgs/Float32`) — 직접 휠 명령 디버그용
- **시리얼**: STS3215 의 기본 baud 인 **1 Mbps** 로 단일 half-duplex 버스. 동일 라인에 좌(ID=1)/우(ID=2) 가 묶여 있으므로 명령은 ID 로 식별
- **노드 파라미터**: `port`(`/dev/servo_port`), `baud`(1_000_000), `left_id`/`right_id` 각각 1/2, `wheel_sep`(m), `wheel_radius`(m), `wheel_mode`(True → wheel mode 선 진입)
- **출력 토픽**: 없음 (시리얼만 송신). 디버깅용 `~battery`, `~feedback` 는 선택적으로 추가 가능.

무엇을 왜 만드는가 — 핵심 설계 의도

- **`/cmd_vel` 을 표준 diff-drive 식으로 좌/우 휠 속도로 변환**합니다. AMR 의 `linear.x`, `angular.z` 를 받아 `wheel_radius`(R), `wheel_sep`(L) 로 다음 두 식을 적용: 좌측 휠 속도 = `(2v - w·L) / (2R)`, 우측 휠 속도 = `(2v + w·L) / (2R)`. 이 식은 차륜형 모바일 로봇의 표준 운동학이며 단계 7의 SLAM Toolbox 가 odom 추정의 기준으로 삼는 값이기도 합니다.
- **wheel mode 진입을 노드 부팅 시점에 강제**합니다. STS3215 의 기본은 위치 모드이므로, 부팅 직후 두 휠 모두에 wheel-mode 전환 명령을 보내고 성공 로그를 남깁니다. 이때 응답이 없으면 e-stop 상태로 진입하지 않고 warn 로그만 흘리고 계속 시도합니다 — 학생이 ID/케이블 문제를 노드 중지 없이 점검할 수 있도록 합니다.
- **동일 이름 publish/subscribe 함정 회피** : 디버깅을 위해 "내가 보낸 명령을 다시 subscribe 로 받고 싶은" 충동을 코드로 직접 옮기면 무한 루프가 됩니다. 이 노드는 `/amr/wheel/debug_actual` 같은 별도 토픽에만 송신하고 `/left_wheel/cmd` 는 receive 만 합니다 (1단계와 같은 함정 — 같은 이름 토픽을 두면 자기로 다시 들어옴).
- **`Send(v)` 헬퍼는 단일 명령 송신 인터페이스**로 둡니다. SDK 메서드명은 버전마다 `WheelMode`/`Mode`/`WheelCommand` 등 명칭이 달라지므로, "속도 정수 하나 → 적절한 SDK 메서드 호출" 인터페이스를 두는 것이 단계 6·7 에서 다른 서보 브랜드로 교체할 때 유리합니다.
- **속도 한계 ±32767 [step/s]** : float 폭주를 막기 위해 subscription 단계에서 ±32767 로 클램프한 뒤 정수형으로 변환해 송신합니다. STS3215 wheel mode 의 데이터시트 범위와 정확히 일치 — SDK 가 별도 클램프를 두지 않으므로 노드 측에서 보장하면 단계 6 의 안전성과 단계 7 의 SLAM 정확성 양쪽에 도움이 됩니다.

흐름도 (요약)

1. `init` 에서 6 개 파라미터 선언 → `st3215.STS3215(port, baud)` 핸들 엽니다.
2. 세 개의 subscription 등록 (`/cmd_vel`, `/left_wheel/cmd`, `/right_wheel/cmd`).
3. `wheel_mode=True` 면 두 서보에 wheel mode 전환 시도.
4. `/cmd_vel` 콜백에서 (v, w) → diff-drive 역학으로 좌/우 휠 속도 산출 → `Send()` 로 송신.
5. `/left/right_wheel/cmd` 콜백은 들어온 `Float32` 값을 그대로 `Send()` 로 송신.
6. 어떤 송신 실패도 warn 로그만 남기고 노드는 결코 죽지 않게 합니다.

`st3215` SDK 사용 시 알아둘 함정

- **메서드명이 패키지/버전별로 다릅니다.** `WheelMode`, `Mode`, `WriteSpec` 같은 후보가 있고, 속도 송신도 `Rotate`, `WheelCommand`, `WritePos` 등의 이름으로 분기됩니다. 가장 안전한 패턴은 "노드 안에서 SDK 메서드명을 import 시점에 확인하고 분기", 또는 PyPI 패키지를 통째로 쓰지 않고 STS3215 datasheet 의 `WRITE` instruction packet 을 직접 인코딩해 `pyserial` 로 송신하는 것입니다. 본 가이드에서는 후자를 권장합니다 — 라이브러리 메서드명에 묶이지 않으므로 단계 7 의 고속 SLAM 루프에서도 안정적입니다.
- **`Ping(ID)` 으로 ID 일치 확인 후 `ChangeID(old, new)`** 로 충돌을 해소합니다. 부록의 점검 스크립트가 이를 위한 1-shot 도구이며, 운영 중 변경은 권장하지 않습니다.

- **half-duplex 라인과 IF 보드의 TX/RX 합선 회로** : Waveshare Bus Servo Driver Board 의 TX ↔ RX 합선 회로는 단방향으로 동작하므로, 단일 호스트가 "송신 → 짧은 지연 → 응답 수신" 패턴을 따르도록 합니다. 동시에 두 호스트가 같은 라인을 점유하면 패킷 충돌이 일어납니다.

직접 구현 체크리스트 (학생용)

- ☐ 6 개 파라미터와 `get_parameter().value` 캐싱 — 1단계 노드와 동일 패턴.
- ☐ `/cmd_vel` 콜백에서 단일 변수 두 개(target_v, target_w) 만 갱신하고 송신은 별도 함수 `Send()` 로 모듈화.
- ☐ diff-drive 식의 부호는 본문 식 (좌=2v−wL, 우=2v+wL) 그대로 — 부호 반대는 차량 거치 방향에 따라 `wheel_radius` 부호 또는 거치 옵션으로 흡수.
- ☐ wheel-mode 전환은 `init` 안에서 1회만 시도 (반복 호출은 서보 인터럽트).
- ☐ 모든 송신은 `int` 로 변환 후 클램프(±32767). float 로 그대로 보내지 않기.
- ☐ 종료 시 핸들 닫기 → 다음 launch 에서 "port already open" 방지.

4.5 `package.xml` & `setup.py`

`src/amr_servo/package.xml`:

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>amr_servo</name>
  <version>0.0.1</version>
  <description>STS3215 wheel-mode controller</description>
  <maintainer email="edu@example.com">Edu</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_python</buildtool_depend>
  <depend>rclpy</depend>
  <depend>std_msgs</depend>
  <depend>geometry_msgs</depend>
  <exec_depend>python3-st3215</exec_depend>
  <exec_depend>python3-serial</exec_depend>
  <export><build_type>ament_python</build_type></export>
</package>
```

`src/amr_servo/setup.py`:

```
from setuptools import setup
package_name = 'amr_servo'
setup(name=package_name, version='0.0.1',
      packages=[package_name],
      data_files=[('share/ament_index/resource_index/packages', ['resource/' + package_name]),
                  ('share/' + package_name, ['package.xml'])],
      install_requires=['setuptools'],
      zip_safe=True, maintainer='Edu', maintainer_email='edu@example.com',
      description='STS3215 wheel-mode controller', license='MIT',
      entry_points={
          'console_scripts': [
              'sts_controller = amr_servo.sts_controller:main',
          ],
      },
)
```

4.5 위치 모드 단독 테스트

```
ros2 topic pub -1 /left_wheel/cmd std_msgs/msg/Float32 '{data: 2048.0}'
ros2 topic pub -1 /left_wheel/cmd std_msgs/msg/Float32 '{data: 1024.0}'  # -90°
```

4.6 TF 와 시각화

Xacro 에서 continuous 조인트로 정의해 둔 `wheel_left_link`, `wheel_right_link` 를 `joint_state_publisher_gui` 로 움직일 수 있습니다.

```
ros2 run joint_state_publisher_gui joint_state_publisher_gui
ros2 run robot_state_publisher robot_state_publisher --ros-args -p robot_description:="$(cat /tmp/amr.urdf)"
ros2 run rviz2 rviz2
```

4.7 트러블슈팅

증상	원인
서보 무반응	12V 외부 전원 미연결 (USB 만 부족)
ID 충돌/잘못 반응	`Ping(ID)` 으로 확인 후 변경 (예: `self.sts.ChangeID(1, 2)`)
half-duplex 송수신 깨짐	IF 보드 우회 (보드 내장 TX ↔ RX 합선 회로가 안전)
보드 자체로 동작하나 PC 코드는 안 잡힘	USB CDC 드라이버(특히 CH343/CH910) 누락 — vendor 드라이버 설치

5단계: PC 통합 테스트 (no-Raspberry Pi)

라즈베리파이를 끼우기 전에 PC 환경에서 **esp32_bridge** + **ldlidar** + **sts_controller** 를 모두 동시에 띄워 **cmd_vel ↔ 서보** + **LiDAR scan 시각화** 까지 일괄 검증합니다.

5.1 통합 launch (`amr_pc_bringup.launch.py`)

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    ld = LaunchDescription()
    # LiDAR (PC USB 포트)
    ld.add_action(IncludeLaunchDescription(
```

```
        PythonLaunchDescriptionSource(
            os.path.join(get_package_share_directory('ldlidar_sl_ros2'),
                         'launch', 'ld14.launch.py')),
        launch_arguments=[('port_name', '/dev/lidar_port'),
                          ('laser_scan_dir', 'True')]))
    # STS3215 controller
    ld.add_action(Node(package='amr_servo', executable='sts_controller',
                       parameters=[{'port': '/dev/servo_port',
                                    'baud': 1_000_000,
                                    'left_id': 1, 'right_id': 2,
                                    'wheel_mode': True}]))
    # ESP32 ↔ PC serial bridge
    ld.add_action(Node(package='amr_serial', executable='esp32_bridge',
                       parameters=[{'port': '/dev/esp32_debug',
                                    'baud': 115200,
                                    'wheel_sep': 0.30,
                                    'wheel_radius': 0.065}]))
    # TF for URDF
    ld.add_action(Node(package='robot_state_publisher',
                       executable='robot_state_publisher',
                       parameters=[{'robot_description': open('/tmp/amr.urdf').read()}]))
    return ld
```

5.2 검증

```
ros2 launch amr_bringup amr_pc_bringup.launch.py
rviz2 & ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

예상 출력:

- `/scan` ⇒ RViz2 에서 레이더가 회전
- 키보드 입력 ⇒ ESP32 가 `$ack,...` echo
- LiDAR 와 모터 동시 동작

5.3 트러블슈팅

- LiDAR 만 보이고 서보 무반응 → 12V 외부 전원 / IF 보드 DC 잭 체크
- ESP32 ack 만 나오고 서보 신호 0 → STS SDK 메서드명 확인 (`WheelCommand`, `Rotate` 등)

6단계: 전체 조립 + ROS teleop

6.1 차량에 조립

1. 양쪽 STS3215 를 정렬 (wheel mode 축)
2. 18650 UPS HAT 을 RPi 40-pin 에 적층 — 별도 5V 출력은 ESP32, LiDAR 등으로 분기
3. ESP32 와 UPS HAT 의 5V/VCC/3.3V 와 공통 GND 확보
4. MPU6050 의 I²C (SDA/SCL) 을 RPi 1/3 (GPIO2/3) 에 연결
5. OLED 의 I²C 0x3C 모듈을 동일 I²C 라인에 부착 (확인 필요: 0x3D 일 가능성)
6. LD14 USB-CP2102 모듈을 RPi USB-A 와 연결

```
[PWR 12V 공급] ─┬─ IF 보드 DC 잭 ──── STS3215 #1, #2
                └─ (옵션) 직접 ESP32 5V로 분기

[18650 (×2)] ── UPS HAT ── RPi 5V (pin 2,4)
```

6.2 모터 ID 매칭

서보 ID 는 기본 1/2 입니다. IF 보드를 PC USB 에 꽂고 **`st3215` 의 `Ping` 으로 확인**하세요. 이 단계는 "ROS 노드가 아니라 PC 단독 1-shot 점검 도구" 인 것이 의도적입니다 — 조립 단계에서 ID 충돌을 빨리 발견하면 6단계 후반의 teleop 실패를 한 번에 줄일 수 있기 때문입니다. 점검 흐름은 다음 세 단계로 충분합니다.

1. **반송(handle) 열기** : `pip3 install st3215` 으로 설치한 SDK 의 `STS3215` 클래스에 `/dev/servo_port` 와 baud `1_000_000` (STS3215 기본 baud) 을 넘겨 객체를 만든다. baud 를 다른 값(예: 115200) 으로 두면 응답이 전혀 오지 않으므로 의심하지 말고 datasheet 의 기본값을 우선 적용한다.
2. **Ping 돌리기** : `Ping(id)` 함수는 짧은 `WRITE` instruction 을 보내고 일정 시간 안에 같은 ID 의 응답 패킷(상태 byte 를 포함) 이 돌아오면 `True` 를 돌려준다. 본 점검에서는 ID 1 부터 4 까지를 한 번씩 호출해 True/False 를 출력만 한다 — 별도의 컬럼을 두지 않고 바로 stdout 으로 흘려도 충분합니다.
3. **판정** : `1, True` 와 `2, True` 만 보이면 정상. 다른 ID 에서 True 가 나오면 라인이 충돌 상태이므로 ID 변경 또는 케이블 점검. `1, False` 면 케이블/전원/IF 보드 결합 순서 의심. **본 절차는 ROS 노드가 동작하기 전 1-shot 으로만 사용**해야 합니다.

   같은 흐름을 ROS 2 노드 안에서 진단 토픽으로 띄우려면 `diagnostic_msgs/DiagnosticArray` 에 상태를 채워 주기 publish 해야 합니다 — 1-shot 점검과 운영 진단은 별개 인터페이스라는 점을 학생에게 분명히 가르칩니다.

장착 시의 ID 재할당 : 서보의 기본 ID 가 다른 경우(예: 둘 다 1), 한쪽을 별도로 IF 보드 단독 연결 + `st3215` 의 `ChangeID(old_id, new_id)` 로 2 로 다시 할당합니다. 이때는 다른 한쪽 서보는 **물리적으로 라인에서 분리**해 두지 않으면 두 서보가 동시에 명령을 받아 ID 충돌이 일어납니다.

6.3 ROS teleop

PS4 XBLS/PS3 컨트롤러를 사용하면 ROS 2 `joy` + `teleop_twist_joy` (ROS Index page) 패키지에서 제공하는 ps3 / ps3-holonomic / xbox config 로 바로 매핑됩니다.

```
sudo apt install -y ros-$ROS_DISTRO-joy ros-$ROS_DISTRO-teleop-twist-joy
ros2 run joy joy_node
ros2 run teleop_twist_joy teleop_node --ros-args \
  -p joy_config:=xbox -p require_enable_button:=true
```

또는 키보드:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

6.4 통합 launch 순서

`amr_bringup/launch/amr_full.launch.py`:

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_bringup = get_package_share_directory('amr_bringup')
    pkg_lidar   = get_package_share_directory('ldlidar_sl_ros2')
    return LaunchDescription([
        # 1) LiDAR
        IncludeLaunchDescription(PythonLaunchDescriptionSource(
            os.path.join(pkg_lidar, 'launch', 'ld14.launch.py')),
            launch_arguments=[('port_name', '/dev/lidar_port'),
                              ('laser_scan_dir', 'True')]),
        # 2) Servo controller (RPi 사이 ttyUSB1 = IF 보드 USB 두번째 라인 / 또는 USB-to-Serial)
        Node(package='amr_servo', executable='sts_controller',
             parameters=[{'port': '/dev/ttyUSB2',
                          'baud': 1_000_000,
                          'left_id': 1, 'right_id': 2,
                          'wheel_mode': True}]),
        # 3) ESP32 bridge to RPi
        Node(package='amr_serial', executable='esp32_bridge',
             parameters=[{'port': '/dev/ttyAMA0', 'baud': 115200,
                          'wheel_sep': 0.30, 'wheel_radius': 0.065}]),
        # 4) TF/URDF
        Node(package='robot_state_publisher', executable='robot_state_publisher',
             parameters=[{'robot_description': open('/tmp/amr.urdf').read()}]),
        # 5) Controller
        Node(package='joy', executable='joy_node'),
        Node(package='teleop_twist_joy', executable='teleop_node',
             parameters=[{'joy_config': 'xbox',
                          'require_enable_button': True}]),
    ])
```

6.5 검증 (체크리스트)

- ☐ 조이스틱 VRx/VRy 가 `cmd_vel.linear.x`(전진), `cmd_vel.angular.z`(회전)으로 표시 (`ros2 topic echo /cmd_vel`)
- ☐ ESP32 가 `$ack,v=…` 출력
- ☐ STS3215 가 wheel mode 로 회전 (바퀴 정방향)
- ☐ `/scan` 이 10 Hz 내외로 발행
- ☐ RViz2 에서 base_link → laser_frame TF 연결 표시

7단계: SLAM

7.1 slam_toolbox 설치

출처: slam_toolbox Jazzy packages docs

```
sudo apt install -y ros-$ROS_DISTRO-slam-toolbox
```

7.2 launch 두 가지 비교

variant	사용 상황	launch	비고
`online_sync`	정적이고 잡음 적은 환경	`online_sync_launch.py`	정확도 높음, 시간지연 큼
`online_async`	동적 환경, 빠른 응답	`online_async_launch.py`	일반적인 AMR 환경 권장
`lifelong`	이미 만들어진 지도를 새 환경에 맞춰 갱신	`lifelong_launch.py`	본 단계에서는 사용 안함

7.3 파라미터 파일 (예시)

`src/amr_bringup/config/mapper_params_online_async.yaml`:

```
slam_toolbox:
  ros__parameters:
    slam_mode: online_async
    topic_scan: /scan
    odom_frame: odom
    map_frame: map
    base_frame: base_link
    resolution: 0.05
    max_laser_range: 12.0          # LD14 datasheet (확인 필요)
    minimum_travel_distance: 0.5
    minimum_travel_heading: 0.5
    use_scan_matching: true
    use_loop_closure: true
    transform_publish_period: 0.05
```

> **odom 발행을 어디서 할 것인가**: 단순화하려면 `/diff_drive_controller/odom` 같은 표준 토픽이 필요합니다. 본 가이드의 단계에서는 (a) `robot_localization` 패키지의 EKF 로 IMU+encoder 융합, 또는 (b) 단순히 LiDAR-only SLAM 으로 SLAM Toolbox 의 내부 odom 추정 사용.
> 본 예제는 **odom 프레임을 비워 두고**, slam_toolbox 가 자체적으로 map → base_link 변환을 publish 하는 것으로 충분합니다 (LiDAR-only 이므로 IMU 보정 미사용; 동시 보정 필요시 `robot_localization` 추가).

7.4 실행

```
ros2 launch amr_bringup amr_full.launch.py
ros2 launch slam_toolbox online_async_launch.py \
  params_file:=~/ros2_amr_ws/src/amr_bringup/config/mapper_params_online_async.yaml
```

7.5 지도 저장

```
# 사용 navigator2 가 설치돼 있으면
# map 도 공통 명령
ros2 run nav2_map_server map_saver_cli -f ~/maps/amr_floor
```

생성된 `~/maps/amr_floor.pgm` / `.yaml` 확인.

7.6 트러블슈팅

증상	점검
`/map` 토픽이 안 보임	①`topic_scan=/scan` 일치 ② base_frame 일치
SLAM 발산(`set poses` 무한 변화)	odom 프레임을 안정화 하거나, LiDAR only 환경이면 `transform_publish_period=0.05` 튜닝
IMU 깜빡임	지금 단계에서는 EKF 미사용;`robot_localization` 의 `two_d_mode:=true` 권장 [자세한 사항 검토 생략]
scan frame 불일치	`base_link → laser_frame` 정적 TF 가 publish 되고 있는지 `ros2 run tf2_tools view_frames.py` 로 확인

8단계: 18650 UPS HAT 전원 시스템

8.1 선택 보드 (예시)

- Waveshare UPS HAT (wiki) — 18650 × 2, 5V 출력 (`up to 5A`), RPi 5/4B/3B+
- 충전 중에도 5V 출력 유지 (입력이 없을 시 배터리로 즉시 전환)
- **PogoPin** 으로 RPi GPIO 의 일부에 전원 액세스 (확인 필요: 모델에 따라 explicit PogoPin 채널이 다름)

8.2 설치

```
[RPi 5/4B/3B+]
   │ (40-pin GPIO)
   ▼
[UPS HAT]  ← 18650 셀 × 2 (보드 자체 거치대 또는 외장 battery expansion)
   │
   ├── 5V out → ESP32 VCC
   ├── 5V out → LD14 모듈 (USB-to-Serial)
   └── (옵션) LiPo charger 전원 입력 via microUSB
```

전원 ON 흐름:

1. UPS HAT 자체 전원 입력(`5V/2A`) 연결
2. 충전 LED/WRITE 유지
3. RPi boot → HDMI/SSH 로 로그인
4. UPS HAT 입력 차단 → 즉시 18650 로 전환되는지 검증

8.3 보호 기능

- **BMS / 보호회로:** 18650 셀은 4.2V 만충 / 3.0V 방전 이르는 모델을 권장. 보호회로가 **기본 내장된** 셀을 사용해야 함 (Large-battery guide)
- 전기적 보호(과전류/저전압): UPS HAT 측 추가 (Waveshare 모델: 약 3A 안전)
- 안전한 셀 사용: 손상된 셀/저가 셀은 **즉시 폐기**

8.4 UPS HAT 배터리 정보 모니터링

항목	가능 여부	비고
5V 출력 정상	시각적으로 LED + RPi boot 유지	–
충전 상태 LED (충전/만충)	표시	–
디지털 잔량/온도	I²C 0xNN 으로 일부 HAT에서 노출	**확인 필요:** 정확한 I²C 주소는 모델/펌웨어에 따라 다름. 보통 0x36/0x40/0x77 영역
시스템 클리어	RPi GPIO interrupt로 low battery 이벤트 받기	**확인 필요:** IRQ/Alert 라인 사용 여부

ROS 2 노드화: `battery_monitor` 패키지로 주기 발행 (`sensor_msgs/BatteryState` 의 `voltage`, `current`, `percentage` 등).

8.5 안전 가이드

- USB-to-Serial 모듈 / ESP32 / 라이다 모두 **공통 GND** 이어야 합니다.
- 충전 중에 즉시 정전 테스트를 시도할 때 ESP32 / 서보 IF 보드의 power가 다시 인가되는지 확인.
- 보호회로 없는 18650 셀은 절대 사용 금지.
- 동시 손상 셀 + 접근 단락 (단자 키스/면 손상) ⇒ 즉시 분리, 가연성 물질과 멀리.

부록 A. Pin Map 요약

기능	RPi (BCM)	ESP32	LiDAR/IF/IMU	비고
Main UART TX (→ ESP32 RX2)	GPIO14	GPIO16	–	RPi ↔ ESP32 (UART2)
Main UART RX (→ ESP32 TX2)	GPIO15	GPIO17	–	동상

기능	RPi (BCM)	ESP32	LiDAR/IF/IMU	비고
Debug UART (USB-to-Serial)	–	GPIO16/17 T-탭	CP2102 listen-only	동시 로그 (확인 필요)
USB-Serial to LD14 (CP2102)	USB-A	–	CP2102 ↔ LD14	`/dev/lidar_port` 고정
USB to STS3215 IF 보드	USB-A	–	USB-C ↔ IF 보드	`/dev/servo_port` 고정
I²C SDA/SCL (MPU6050, OLED)	GPIO2 / GPIO3	–	MPU6050(0x68), OLED(0x3C)	OLED 0x3D 모델 확인 필요
5V output	pin 2/4 (UPS HAT)	5V	서보 / LD14 모듈 전원	UPS HAT 출력
GND	pin 6, 9, 14…	GND	공통	–

부록 B. 토픽 / 서비스 정리

Name	Type	Publisher	Subscriber	비고
`/cmd_vel`	`geometry_msgs/Twist`	teleop	`esp32_bridge`, `slam_toolbox`	조이스틱/키보드
`/amr/status`	`std_msgs/String`	`esp32_bridge`	debug GUI	`$status,...` 라인 echo
`/left_wheel/cmd`	`std_msgs/Float32`	diff_drive ad-hoc	`sts_controller`	wheel mode 속도
`/right_wheel/cmd`	`std_msgs/Float32`	diff_drive ad-hoc	`sts_controller`	동상
`/scan`	`sensor_msgs/LaserScan`	`ldlidar_node`	`slam_toolbox`, RViz	LD14 10 Hz
`/map`	`nav_msgs/OccupancyGrid`	`slam_toolbox`	`rviz`	SLAM 시각화
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	`robot_state_publisher`, `slam_toolbox`	`rviz`	base_link ↔ laser, map → odom → base_link
`~battery` (옵션)	`sensor_msgs/BatteryState`	(직접 작성)	RViz/dashboard	UPS HAT 잔량**(I²C addr 확인 필요)**

부록 C. 미검증 항목 / 추가 확인 필요

항목	상태	권장 다음 액션
IPU 의 정확한 의미	사용자 명세에 등장했으나 정확한 정의 미확인	멘토/제작자에게 명확화 요청. 본 가이드에서는 `cmd_vel` 표준 흐름으로 우회
STS IF 보드 정확 모델/SKU	"STS IF 보드" 만 표기됨	구매처 박스 라벨로 모델명 확인 (Waveshare "Serial Bus Servo Driver Board" 또는 호환 SmartElex 등)
ESP32 의 "시리얼 2 번" 가 어떤 채널인지	사용자: "디버깅 시리얼 2" 정의 모호	ESP32 UART0 (USB-UART) 가 디버깅 기본. UART2를 메인으로 옮추는 절충안을 본 단계에 표기
ESP32 UART 안정 핀 (SoftSerial)	GPIO 가능하지만 baud 115kbps+ 한계	디버깅 시점에는 UART2-tap 단독 listen-only 권장
CP2102 idVendor (10c4/ea60)	LD14 사용 예제지만 본인 모듈 다를 수 있음	`lsusb -v` 로 실제 idVendor/ProductId 확인
Waveshare UPS HAT I²C address	잔량/충전 상태 노출이 모델별 상이	제조사 wiki 의 schematic 확인 후 노드 작성
OLED (0.96") I²C addr	일부 0x3C / 일부 0x3D	`i2cdetect -y 1` 으로 본인 모듈 해소
STS3215 wheel mode API 메서드명	`st3215` PyPI 패키지의 명칭이 출시/버전에 따라 `Rotate`/`WheelMode`/`WheelCommand` 등 다양	결고: 정식 STS3215 datasheet 의 `WRITE` 패킷 직접 인코딩 → 안전
LD14 max laser_range	일반적으로 8–12 m 영역, 정확한 모델별 명세	사용자 박스 라벨/스펙 시트 확인

마무리

이 가이드는 "복잡한 패키지를 통째로 쓰지 않고, **ROS 2 노드를 직접 작성** 하면서 *바닥부터 차량을 살려 나가는 경험* 을 학습자에게 제공하기 위해 작성되었습니다. 단계 끝의 **트러블슈팅 표** 가 비어 보일수록 좋은 것 — 그리고 비어 보이지 않는 부분이 다음 학습자/멘토가 개선할 부분입니다.

참조 자료

명령에 따라 검증 출처를 본문에 인라인으로 노출했습니다. 주요 자료:

- ROS2 Jazzy Releases: https://docs.ros.org/en/jazzy/Releases.html
- ROS2 EOL tracker: https://eosl.date/eol/product/ros-2/
- ldlidar_sl_ros2 (LD14 공식): https://github.com/ldrobotSensorTeam/ldlidar_sl_ros2
- ST3215 (STS3215 호환) — DFRobot: https://www.dfrobot.com/product-2962.html
- ST3215 — Waveshare Wiki: https://www.waveshare.com/wiki/ST3215_Servo
- Waveshare Serial Bus Servo Driver Board wiki: https://www.waveshare.com/wiki/Bus_Servo_Driver_Board
- ESP32 pinout (RandomNerd): https://randomnerdtutorials.com/esp32-pinout-reference-gpios/
- ESP32 UART default pins (PCBSync): https://pcbsync.com/esp32-pinout-reference/
- MPU6050 I²C 0x68/0x69: https://controllerstech.com/mpu6050-arduino-tutorial/
- Oceanlabz ESP32 UART default: https://www.oceanlabz.in/esp32-pinout-reference/
- Reddit STS3215 forum pin: https://forum.arduino.cc/t/control-servo-sts3215-with-esp32-wroom-32d/1393738
- eevblog Waveshare driver thread: https://www.eevblog.com/forum/projects/trying-to-understand-waveshare-servo-driver-with-esp32/
- 18650 가이드 (Large-battery): https://www.large-battery.com/blog/3-7v-lithium-ion-battery-characteristics-usage-18650-safety/
- Waveshare UPS HAT: https://www.waveshare.com/ups-hat.htm
- slam_toolbox Jazzy: https://docs.ros.org/en/jazzy/p/slam_toolbox/
- teleop_twist_joy: https://index.ros.org/r/teleop_twist_joy/

- st3215 PyPI: https://pypi.org/project/st3215/